

CS4031 — Compiler Construction
Assignment 02

LL(1) Parser Design & Implementation

Field	Details
Students	23i-0563 23i-0713
Section	D
Language	C++17
Semester	Spring 2026
Submission	23i_0563-23i-0713-D.zip

1. Introduction

LL(1) parsing is a top-down parsing technique used in the front-end of compilers. The name means it reads input from **Left** to right, uses a **Leftmost** derivation, and looks ahead **1** token to make every parsing decision deterministically. It is the foundation of predictive parsing and recursive-descent compilers.

This assignment implements a complete LL(1) parser pipeline in C++17 that: (1) reads any Context-Free Grammar (CFG) from a text file, (2) eliminates left factoring and both direct and indirect left recursion, (3) computes FIRST and FOLLOW sets, (4) constructs the LL(1) parsing table and detects conflicts, (5) parses input strings with a stack-based algorithm, (6) recovers from errors using panic-mode recovery, and (7) generates ASCII parse trees for accepted strings.

2. Approach

2.1 Data Structures

Structure	File	Purpose
struct Production	grammar.h	Stores lhs (string) + rhs (vector<string>) for one rule
class Grammar	grammar.h/cpp	Vector of Productions; owns all transformations
class FirstFollow	first_follow.h/cpp	Holds firstSets / followSets as vector<vector<string>>
class Stack	stack.h/cpp	Custom array-backed stack (push/pop/top/isEmpty)
struct TreeNode	tree.h/cpp	N-ary tree node with label + vector of children
class ParseTree	tree.h/cpp	Root pointer + ASCII printer (unicode box-drawing)
class Parser	parser.h/cpp	2-D table[NT][term], conflict table, parse() method
class ErrorHandler	error_handler.h/cpp	Collects ParseError structs, reports with line/col

2.2 Algorithm Pipeline

- **Step 1. Load Grammar** — Grammar::loadFromFile() tokenises each line on '|' and '->' into Productions.
- **Step 2. Left Factoring** — applyLeftFactoring() iterates until no two productions for the same NT share a first token. A new NT A' is introduced for the suffix alternatives.
- **Step 3. Left Recursion Removal** — removeLeftRecursion() follows the standard $O(n^2)$ algorithm: for each NT A_i , indirect productions $A_i \rightarrow A_j \gamma$ ($j < i$) are expanded by substituting all A_j productions; then direct recursion is eliminated via removeDirectForSpecificNT().
- **Step 4. FIRST Sets** — computeFirst() iterates to fixed point. For every production $A \rightarrow Y_1 \dots Y_n$ it adds $\text{FIRST}(Y_1) \setminus \{\epsilon\}$ and, if Y_1 can derive ϵ , continues to Y_2 , etc. If all symbols can derive ϵ , ϵ is added to $\text{FIRST}(A)$.
- **Step 5. FOLLOW Sets** — computeFollow() iterates to fixed point. $\$$ is seeded into $\text{FOLLOW}(\text{start})$. For each occurrence of NT B in a RHS, FIRST of the remaining string (minus ϵ) is added to $\text{FOLLOW}(B)$, and if that string can be ϵ , FOLLOW of the LHS is also added.

- **Step 6. Table Construction** — `buildTable()` fills $M[A, a]$ for each production. Conflicts (multiple entries) are recorded; the grammar is flagged as non-LL(1).
- **Step 7. Stack-Based Parsing** — `parse()` initialises $[\$ S]$ on the stack. It loops: match terminals, expand non-terminals via the table, or trigger error recovery.
- **Step 8. Panic-Mode Recovery** — On an empty table cell, $\text{FOLLOW}(\text{top-NT}) \cup \{\$\}$ is the sync set. Stack symbols are popped and input tokens skipped until a sync symbol is found. A force-pop prevents infinite loops.
- **Step 9. Parse Tree** — Each Predict action attaches child `TreeNode`s to the current node. Accepted strings are printed as unicode ASCII trees.

2.3 Design Decisions

- All classes are in **separate .h / .cpp** file pairs as required.
- New non-terminals use the **prime (')** suffix (e.g., `Expr'`), chaining primes if the name already exists.
- The parser table stores **all conflict production indices** (not just the first), enabling accurate LL(1) conflict reporting.
- The `Stack` class is a standalone wrapper — not `std::stack` — so its `toString()` for trace display is trivial.
- The parse tree is built **incrementally** during parsing via a parallel `nodeStack` that mirrors the parse stack.
- Error recovery guarantees **termination**: if neither the stack nor the input index changes after `panicRecover()`, the top symbol is force-popped.

2.4 Indirect Left Recursion Handling

The standard algorithm orders all non-terminals $A_1 \dots A_n$. For each A_i , it substitutes any production $A_i \rightarrow A_j \gamma$ (where $j < i$) with the full expansion of A_j 's productions before eliminating remaining direct left recursion. This is implemented in **`Grammar::removeLeftRecursion()`**. Grammar 4 (Prog/Stmt/Expr) demonstrates this: `Expr` has direct left recursion which is fully eliminated.

2.5 Error Recovery Strategy — Panic Mode

On detecting an error (empty table cell, terminal mismatch, or premature end), the parser calls **`panicRecover()`**:

- Compute sync set = $\text{FOLLOW}(\text{top non-terminal}) \cup \{\$\}$.
- Pop stack symbols while the current input token is not in FOLLOW of the new top.
- Skip input tokens until a sync symbol is reached.
- If zero progress was made (both stack size and input index unchanged), force-pop the top to guarantee termination.
- Resume parsing — further errors in the same string are detected and reported.

4. Test Cases

4.1 Grammar 1 — Simple Grammar with Epsilon

```
Start -> First Second
First -> a | epsilon
Second -> b
```

input/grammar1.txt — No transformations needed

Input String	Expected	Result	Notes
a b	ACCEPT	ACCEPT	First=a, Second=b
b	ACCEPT	ACCEPT	First derives epsilon
a b	ACCEPT	ACCEPT	Duplicate valid string
a	REJECT	REJECT	Missing Second (b)
b b	REJECT	REJECT	Extra b after Second
a a b	REJECT	REJECT	Two a tokens, only one expected
c	REJECT	REJECT	Unknown terminal c

4.2 Grammar 2 — Expression Grammar (Left Recursion)

```
Expr -> Expr + Term | Term
Term -> Term * Factor | Factor
Factor -> ( Expr ) | id
```

input/grammar2.txt — Direct left recursion removed from Expr and Term

After left recursion removal:

```
Expr  -> Term Expr'
Expr' -> + Term Expr' | epsilon
Term  -> Factor Term'
Term' -> * Factor Term' | epsilon
Factor -> ( Expr ) | id
```

Input String	Expected	Result	Notes
id + id * id	ACCEPT	ACCEPT	Standard arithmetic, * binds tighter
(id + id) * id	ACCEPT	ACCEPT	Parenthesised sub-expression

Input String	Expected	Result	Notes
id * id + id	ACCEPT	ACCEPT	Left-to-right with precedence
id	ACCEPT	ACCEPT	Single terminal
(id + id)	ACCEPT	ACCEPT	Parenthesised expression
id * (id + id)	ACCEPT	ACCEPT	Mixed
id + id * id	ACCEPT	ACCEPT	Same as line 1
id + * id	REJECT	REJECT	Missing operand after +
(id + id	REJECT	REJECT	Unclosed parenthesis
id id	REJECT	REJECT	Two ids with no operator
+ id	REJECT	REJECT	Starts with operator
id +	REJECT	REJECT	Trailing operator
()	REJECT	REJECT	Empty parentheses

4.3 Grammar 3 — Statement Grammar (Left Factoring / Dangling Else)

```

Stmt -> if Cond then Stmt else Stmt | if Cond then Stmt | assign
Cond -> id

```

input/grammar3.txt — Left factoring applied; grammar is NOT LL(1) (dangling else conflict)

The dangling-else ambiguity creates a conflict at $M[StmtFact..., else]$ since both 'else Stmt' and epsilon are reachable. The parser correctly detects and reports this as a non-LL(1) conflict. Parsing proceeds using the first-production rule (greedy else association).

Input String	Expected	Result	Notes
assign	ACCEPT	ACCEPT	Minimal statement
if id then assign	ACCEPT	ACCEPT	If without else
if id then assign else assign	ACCEPT	ACCEPT	If-then-else
if id then if id then assign	ACCEPT	ACCEPT	Nested if
if then assign	REJECT	REJECT	Missing condition
if id assign	REJECT	REJECT	Missing 'then'
if id then	REJECT	REJECT	Missing body
else assign	REJECT	REJECT	Else without if

4.4 Grammar 4 — Program Grammar (Left Recursion in Expr)

```

Prog -> Stmt Prog | epsilon
Stmt -> id assign Expr | print Expr
Expr -> Expr plus Term | Term
Term -> num | id

```

input/grammar4.txt — Left recursion in Expr removed

Input String	Expected	Result	Notes
id assign num	ACCEPT	ACCEPT	Simple assignment
print num	ACCEPT	ACCEPT	Print statement
id assign id plus num	ACCEPT	ACCEPT	Assignment with expression
id assign num print id	ACCEPT	ACCEPT	Two statements
assign num	REJECT	REJECT	Missing lhs id
id id	REJECT	REJECT	Missing assign
print	REJECT	REJECT	Missing expression after print
id assign	REJECT	REJECT	Missing rhs
num plus id	REJECT	REJECT	Expr not a statement

5. Verification

- **Manual trace cross-check:** The parsing trace for 'id + id * id' (Grammar 2) was compared step-by-step against the textbook LL(1) algorithm. All 17 steps match.
- **FIRST/FOLLOW cross-check:** FIRST and FOLLOW sets for Grammar 2 were computed by hand and match the program output exactly.
- **Conflict detection:** Grammar 3 is known to be ambiguous (dangling else). The parser correctly reports a conflict and flags it as non-LL(1).
- **Error recovery continuity:** Multiple error inputs were tested to confirm the parser continues after the first error and reports all subsequent errors.
- **Empty input:** An empty line produces no tokens and is handled gracefully.
- **Valgrind:** Ran with **make valgrind** — zero memory leaks reported on all test grammars.

6. Sample Outputs

6.1 Grammar 2 — FIRST and FOLLOW Sets

```

FIRST(Expr)   = { (, id }
FIRST(Expr')  = { +, epsilon }
FIRST(Term)   = { (, id }
FIRST(Term')  = { *, epsilon }
FIRST(Factor) = { (, id }

FOLLOW(Expr)  = { $, ) }
FOLLOW(Expr') = { $, ) }
FOLLOW(Term)  = { +, $, ) }
FOLLOW(Term') = { +, $, ) }
FOLLOW(Factor) = { *, +, $, ) }

```

6.2 Parsing Trace — 'id + id * id'

Step	Stack (bottom->top)	Input Remaining	Action
1	\$ Expr	id + id * id \$	Predict: Expr -> Term Expr'

2	\$ Expr' Term	id + id * id \$	Predict: Term -> Factor Term'
3	\$ Expr' Term' Factor	id + id * id \$	Predict: Factor -> id
4	\$ Expr' Term' id	id + id * id \$	Match 'id'
5	\$ Expr' Term'	+ id * id \$	Predict: Term' -> epsilon
6	\$ Expr'	+ id * id \$	Predict: Expr' -> + Term Expr'
7	\$ Expr' Term +	+ id * id \$	Match '+'
...	...	...	...
17	\$	\$	Accept

6.3 Error Recovery — 'id + * id'

```

8      $ Expr' Term          * id $
[ERROR] Line 1, Col 3: No rule for non-terminal 'Term' with lookahead '*'.
[RECOVERY] Sync set for Term: { + $ ) }
[RECOVERY] Popping 'Term' from stack
[RECOVERY] Popping 'Expr'' from stack
[RECOVERY] Skipping input token '*'
[RECOVERY] Skipping input token 'id'
Result: *** STRING REJECTED *** (1 error(s))

```

6.4 Parse Tree — 'id + id * id'

```

Expr
+--> Term
|
| +--> Factor
| | +---> id
| +--> Term' (epsilon)
+--> Expr'
|
| +--> +
| +--> Term
| | +--> Factor --> id
| | +--> Term'
| | | +--> *
| | | +--> Factor --> id
| | | +--> Term' (epsilon)
+--> Expr' (epsilon)

```

7. Conclusion

This assignment provided hands-on experience with every stage of a predictive parser: grammar normalisation (left factoring and left recursion removal), FIRST/FOLLOW computation, LL(1) table construction with conflict detection, and robust stack-based parsing with panic-mode error recovery and parse tree generation.

Key lessons learned: (1) Epsilon handling is a surprisingly subtle source of bugs — every algorithm step must correctly propagate the 'can derive epsilon' property. (2) Error recovery must be designed to guarantee termination; a force-pop mechanism is essential when the sync set contains only end-of-input markers. (3) Separating concerns into distinct classes (Grammar, FirstFollow, Stack, Parser, ParseTree, ErrorHandler) made testing and debugging each component in isolation straightforward. (4) LL(1) grammars are a strict subset of CFGs — the dangling-else ambiguity in Grammar 3 is a real-world reminder that not every natural language construct maps cleanly to LL(1).

The complete implementation passes all 8 test runs (4 grammars x valid/error inputs), produces correct FIRST/FOLLOW sets, generates readable parse trees, and compiles with zero warnings

under **g++ -std=c++17 -Wall -Wextra.**